

---

# 심층 합성곱 신경망을 이용한 ImageNet 분류

---

Alex Krizhevsky  
토론토 대학교  
kriz@cs.utoronto.ca

Ilya Sutskever  
토론토 대학교  
ilya@cs.utoronto.ca

Geoffrey E. Hinton  
토론토 대학교  
hinton@cs.utoronto.ca

## 초록

우리는 ImageNet LSVRC-2010 대회의 120만 장 고해상도 이미지를 1,000개의 서로 다른 클래스로 분류하기 위해 크고 깊은 합성곱 신경망을 학습했다. 테스트 데이터에서 top-1과 top-5 오류율은 각각 37.5%와 17.0%였으며, 이는 이전의 최고 성능보다 크게 우수하다. 이 신경망은 약 6천만 개의 파라미터와 65만 개의 뉴런을 가지며, 5개의 합성곱 레이어(일부 뒤에는 max-pooling 레이어가 위치함)와 3개의 완전 연결 레이어, 그리고 마지막 1,000-way softmax로 구성된다. 학습을 빠르게 하기 위해 비포화 뉴런과 매우 효율적인 GPU 기반 합성곱 구현을 사용했다. 완전 연결 레이어의 과적합을 줄이기 위해 최근 제안된 정규화 기법인 “dropout”을 사용했으며, 매우 효과적임을 확인했다. 또한 이 모델의 변형을 ILSVRC-2012 대회에 출전시켜 top-5 테스트 오류율 15.3%로 우승했으며, 2위의 오류율은 26.2%였다.

## 1 서론

현재의 객체 인식 방법은 머신러닝 기법을 핵심적으로 활용한다. 성능을 높이기 위해 더 큰 데이터셋을 수집하고, 더 강력한 모델을 학습하며, 과적합을 방지하는 더 나은 기법을 사용할 수 있다. 최근까지 라벨이 있는 이미지 데이터셋은 수만 장 규모로 비교적 작았다(예: NORB [16], Caltech-101/256 [8, 9], CIFAR-10/100 [12]). 이런 규모의 데이터셋으로도 단순한 인식 문제는 상당히 잘 풀 수 있으며, 특히 라벨을 보존하는 변환으로 데이터를 증강하면 더 그렇다. 예를 들어 MNIST 숫자 인식의 당시 최고 오류율(<0.3%)은 사람의 성능에 근접한다 [4]. 그러나 실제 환경의 객체는 매우 다양한 형태로 나타나므로 이를 인식하도록 학습하려면 훨씬 큰 학습 집합이 필요하다. 작은 이미지 데이터셋의 한계는 널리 알려져 있었지만(예: Pinto et al. [21]), 수백만 장 규모의 라벨 데이터셋을 수집할 수 있게 된 것은 비교적 최근이다. 새로운 대규모 데이터셋에는 수십만 장의 완전 분할 이미지를 포함하는 LabelMe [23]와, 22,000개가 넘는 범주에 1,500만 장 이상의 라벨된 고해상도 이미지를 포함하는 ImageNet [6]이 있다.

수백만 장의 이미지에서 수천 종류의 객체를 학습하려면 학습 용량이 큰 모델이 필요하다. 그러나 객체 인식 문제의 복잡성은 ImageNet처럼 큰 데이터셋만으로도 완전히 규정할 수 없을 정도이므로, 부족한 데이터를 보완할 수 있는 많은 사전 지식도 모델에 포함되어야 한다. 합성곱 신경망(CNN)은 이러한 모델의 한 종류다 [16, 11, 13, 18, 15, 22, 26]. CNN의 용량은 깊이와 너비를 조절해 제어할 수 있으며, 이미지의 성질, 즉 통계의 정상성(stationarity)과 픽셀 의존성의 국소성에 대해 강하고 대체로 타당한 가정을 둔다. 따라서 비슷한 크기의 레이어를 가진 일반적인 순방향 신경망과 비교하면 CNN은 연결과 파라미터 수가 훨씬 적어 학습이 쉬우면서도, 이론적으로 가능한 최고 성능은 약간만 낮을 것으로 기대된다.

CNN은 이러한 매력적인 특성과 국소 구조의 상대적 효율성에도 불구하고, 고해상도 이미지에 대규모로 적용하기에는 여전히 계산 비용이 지나치게 컸다. 다행히 오늘날의 GPU는 고도로 최적화된 2차원 합성곱 구현과 결합하면 충분히 큰 CNN을 학습할 수 있을 만큼 강력하며, ImageNet 같은 최신 데이터셋에는 심각한 과적합 없이 이러한 모델을 학습시킬 수 있을 정도로 많은 라벨 예제가 포함되어 있다.

이 논문의 구체적인 기여는 다음과 같다. 우리는 ILSVRC-2010과 ILSVRC-2012 대회에서 사용된 ImageNet 부분집합을 대상으로 당시 가장 큰 수준의 합성곱 신경망 중 하나를 학습하여, 해당 데이터셋에서 이전에 보고된 결과를 크게 뛰어넘는 성능을 달성했다. 또한 2차원 합성곱과 CNN 학습에 필요한 다른 모든 연산을 위한 고도로 최적화된 GPU 구현을 작성해 공개했다<sup>1</sup>. 우리 네트워크에는 성능을 높이고 학습 시간을 줄이는 새롭거나 비일반적인 여러 특징이 있으며, 이는 3절에서 설명한다. 네트워크의 크기가 매우 커서 120만 개의 라벨 학습 예제가 있음에도 과적합이 큰 문제가 되었고, 이를 막기 위한 여러 효과적인 기법을 사용했으며 4절에서 설명한다. 최종 네트워크는 5개의 합성곱 레이어와 3개의 완전 연결 레이어로 이루어진다. 이 깊이는 중요한 것으로 보인다. 각 합성곱 레이어는 전체 파라미터의 1% 이하만 포함하지만, 그중 하나라도 제거하면 성능이 저하됨을 확인했다.

결국 네트워크의 크기는 주로 당시 GPU가 제공하는 메모리와 우리가 감수할 수 있는 학습 시간에 의해 제한된다. 우리 네트워크는 GTX 580 3GB GPU 두 장에서 학습하는 데 5~6일이 걸렸다. 모든 실험은 더 빠른 GPU와 더 큰 데이터셋을 사용할 수 있게 되면 단순히 규모를 키우는 것만으로도 결과를 더 개선할 수 있음을 시사한다.

## 2 데이터셋

ImageNet은 약 22,000개 범주에 속하는 1,500만 장 이상의 라벨된 고해상도 이미지로 구성된 데이터셋이다. 이미지는 웹에서 수집되었고, Amazon Mechanical Turk 크라우드소싱 도구를 이용해 사람이 라벨을 부여했다. 2010년부터 Pascal Visual Object Challenge의 일부로 ImageNet Large-Scale Visual Recognition Challenge(ILSVRC)가 매년 열렸다. ILSVRC는 1,000개 범주 각각에 약 1,000장의 이미지를 포함하는 ImageNet의 부분집합을 사용한다. 전체적으로 학습 이미지 약 120만 장, 검증 이미지 50,000장, 테스트 이미지 150,000장으로 구성된다.

ILSVRC-2010은 ILSVRC 가운데 테스트 세트의 정답 라벨을 사용할 수 있는 유일한 버전이므로, 대부분의 실험을 이 버전에서 수행했다. 또한 ILSVRC-2012 대회에도 모델을 출전시켰기 때문에 6절에서 이 데이터셋에 대한 결과도 보고한다. 다만 이 버전의 테스트 라벨은 공개되어 있지 않다. ImageNet에서는 일반적으로 top-1과 top-5 두 오류율을 보고한다. top-5 오류율은 모델이 가장 가능성이 높다고 판단한 5개의 라벨 안에 정답 라벨이 포함되지 않은 테스트 이미지의 비율이다.

ImageNet 이미지는 해상도가 제각각이지만, 우리 시스템은 고정된 입력 차원을 요구한다. 따라서 이미지를  $256 \times 256$ 의 고정 해상도로 다윈샘플링했다. 직사각형 이미지의 경우 먼저 짧은 변의 길이가 256이 되도록 크기를 조정하고, 그 이미지의 중앙에서  $256 \times 256$  패치를 잘라냈다. 학습 세트에서 계산한 평균 값을 각 픽셀에서 빼는 것 외에는 다른 전처리를 하지 않았다. 즉, 중심화된 원시 RGB 픽셀 값을 그대로 네트워크 학습에 사용했다.

## 3 아키텍처

우리 네트워크의 아키텍처는 그림 2에 요약되어 있다. 학습되는 레이어는 총 8개이며, 5개는 합성곱 레이어이고 3개는 완전 연결 레이어다. 아래에서는 네트워크 아키텍처의 새롭거나 비일반적인 특징을 설명한다. 3.1~3.4절은 우리가 평가한 중요도 순으로, 가장 중요한 항목부터 배치했다.

<sup>1</sup><http://code.google.com/p/cuda-convnet/>

### 3.1 ReLU 비선형성

뉴런의 출력  $f$ 를 입력  $x$ 의 함수로 모델링하는 표준적인 방법은  $f(x)=\tanh(x)$  또는  $f(x)=(1+e^{(-x)})^{-1}$ 을 사용하는 것이다. 경사하강법의 학습 시간 관점에서 이러한 포화형 비선형 함수는 비포화 비선형 함수  $f(x)=\max(0,x)$ 보다 훨씬 느리다. Nair와 Hinton [20]을 따라 우리는 이 비선형성을 사용하는 뉴런을 Rectified Linear Unit(ReLU)이라 부른다. ReLU를 사용한 심층 합성곱 신경망은  $\tanh$  유닛을 사용한 동등한 네트워크보다 여러 배 빠르게 학습된다. 그림 1은 특정 4레이어 합성곱 신경망이 CIFAR-10에서 학습 오류율 25%에 도달하기까지 필요한 반복 횟수를 보여준다. 이 그래프는 전통적인 포화형 뉴런 모델을 사용했다면 이 연구에서처럼 큰 신경망을 실험하기 어려웠을 것임을 보여준다.

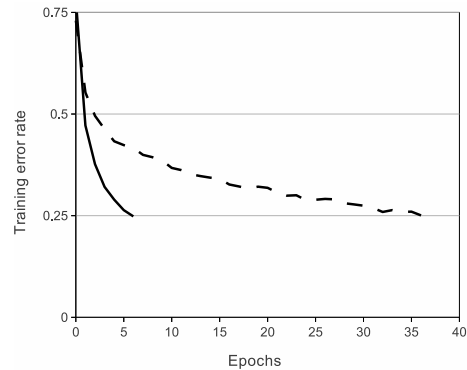


그림 1: ReLU를 사용한 4레이어 합성곱 신경망(실선)은 CIFAR-10에서 학습 오류율 25%에 도달하는 데,  $\tanh$  뉴런을 사용한 동등한 네트워크(점선)보다 약 6배 빠르다. 각 네트워크의 학습률은 가능한 한 빠르게 학습되도록 독립적으로 선택했다. 어떤 정규화도 사용하지 않았다. 여기서 보인 효과의 크기는 네트워크 아키텍처에 따라 달라지지만, ReLU 네트워크는 포화형 뉴런을 사용하는 동등한 네트워크보다 일관되게 여러 배 빠르게 학습된다.

CNN에서 전통적인 뉴런 모델의 대안을 고려한 것은 우리가 처음이 아니다. 예를 들어 Jarrett et al. [11]은 Caltech-101 데이터셋에서 국소 평균 풀링에 앞서 contrast normalization을 적용할 때 비선형 함수  $f(x)=|\tanh(x)|$ 가 특히 잘 작동한다고 보고했다. 그러나 그 데이터셋에서는 과적합 방지가 주된 관심사이므로, 그들이 관찰한 효과는 우리가 ReLU 사용 시 보고하는 학습 세트 적합 속도의 향상과는 다르다. 빠른 학습은 대규모 데이터셋에서 큰 모델을 학습할 때 성능에 매우 큰 영향을 준다.

### 3.2 다중 GPU에서의 학습

GTX 580 GPU 한 장의 메모리는 3GB뿐이므로 학습 가능한 네트워크의 최대 크기에 제한이 있다. 120만 개의 학습 예제는 한 장의 GPU에 들어가지 않을 정도로 큰 네트워크를 학습하기에 충분한 것으로 나타났다. 따라서 우리는 네트워크를 두 GPU에 나누어 배치했다. 당시 GPU는 호스트 메모리를 거치지 않고 서로의 메모리를 직접 접근 쓸 수 있어 GPU 간 병렬화에 특히 적합하다. 우리가 사용한 병렬화 방식은 기본적으로 커널(또는 뉴런)의 절반씩을 각 GPU에 배치하되, GPU끼리는 특정 레이어에서만 통신하도록 하는 추가적인 방법을 사용한다. 예를 들어 3번째 레이어의 커널은 2번째 레이어의 모든 커널 맵을 입력으로 받지만, 4번째 레이어의 커널은 3번째 레이어 중 같은 GPU에 있는 커널 맵만 입력으로 받는다. 연결 패턴의 선택은 교차 검증의 대상이지만, 이 방식은 통신량이 계산량에서 허용 가능한 비율이 될 때까지 정밀하게 조정할 수 있게 해준다.

결과적으로 얻어진 아키텍처는 Cireşan et al. [5]이 사용한 “columnar” CNN과 다소 비슷하지만, 우리의 열(column)은 서로 독립적이지 않다(그림 2 참조). 이 방식은 각 합성곱 레이어의 커널 수를 절반으로 줄여 한 GPU에서 학습한 네트워크와 비교했을 때 top-1과 top-5 오류율을 각각 1.7%와 1.2% 낮춘다. 두 GPU를 사용한 네트워크는 한 GPU 네트워크보다 학습 시간도 약간 짧다<sup>2</sup>.

<sup>2</sup> 실제로 한 GPU 네트워크는 마지막 합성곱 레이어에서 두 GPU 네트워크와 같은 수의 커널을 갖는다. 대부분의 파라미터가 마지막 합성곱 레이어를 입력으로 받는 첫 번째 완전 연결 레이어에 있기 때문이다. 따라서 두 네트워크의 총 파라미터 수를 비슷하게 맞추기 위해 마지막 합성곱 레이어와 그 뒤의 완전 연결 레이어 크기는 절반으로 줄이지 않았다. 그러므로 이 비교는 한 GPU 네트워크가 두 GPU 네트워크의 정확히 “절반 크기”보다 더 크기 때문에 한 GPU 쪽에 유리한 비교다.

### 3.3 국소 응답 정규화

ReLU는 포화를 방지하기 위해 입력 정규화를 필요로 하지 않는다는 장점이 있다. 적어도 일부 학습 예제가 ReLU에 양의 입력을 만들어낸다면 그 뉴런에서는 학습이 일어난다. 그럼에도 우리는 다음의 국소 정규화 방식이 일반화 성능에 도움이 됨을 확인했다. 위치  $(x,y)$ 에서 커널  $i$ 를 적용한 뒤 ReLU 비선형성을 적용해 얻은 뉴런 활성값을  $a_i(x,y)$ 라 할 때, 응답 정규화된 활성값  $b_i(x,y)$ 는 다음 식으로 정의된다.

$$b_{x,y}^i = a_{x,y}^i / \left( k + \alpha \sum_{j=\max(0,i-n/2)}^{\min(N-1,i+n/2)} (a_{x,y}^j)^2 \right)^\beta$$

여기서 합은 동일한 공간 위치에 있는  $n$ 개의 “인접한” 커널 맵에 대해 수행되며,  $N$ 은 해당 레이어의 전체 커널 수다. 커널 맵의 순서는 임의적이며 학습 전에 정해진다. 이러한 응답 정규화는 실제 뉴런에서 관찰되는 측면 억제(lateral inhibition)의 한 형태를 구현하며, 서로 다른 커널로 계산된 뉴런 출력 중 큰 활성값들이 경쟁하도록 만든다. 상수  $k, n, \alpha, \beta$ 는 검증 세트를 이용해 정하는 하이퍼파라미터이며, 우리는  $k=2, n=5, \alpha=10^{-4}, \beta=0.75$ 를 사용했다. 이 정규화는 일부 레이어에서 ReLU 비선형성을 적용한 뒤 수행했다(3.5절 참조).

이 방식은 Jarrett et al. [11]의 국소 대비 정규화(local contrast normalization)와 어느 정도 유사하지만, 평균 활성값을 빼지 않으므로 우리 방식은 “밝기 정규화(brightness normalization)”라고 부르는 편이 더 정확하다. 응답 정규화는 top-1과 top-5 오류율을 각각 1.4%와 1.2% 줄였다. CIFAR-10에서도 효과를 확인했으며, 4레이어 CNN의 테스트 오류율은 정규화 없이 13%, 정규화 사용 시 11%였다<sup>3</sup>.

### 3.4 중첩 풀링

CNN의 풀링 레이어는 같은 커널 맵에서 서로 이웃한 뉴런 집단의 출력을 요약한다. 전통적으로 인접한 풀링 유닛이 요약하는 영역은 서로 겹치지 않는다(예: [17, 11, 4]). 좀 더 정확히 말하면, 풀링 레이어는  $s$  픽셀 간격으로 배치된 풀링 유닛의 격자로 볼 수 있고, 각 유닛은 자신의 위치를 중심으로  $z \times z$  크기의 이웃 영역을 요약한다.  $s=z$ 로 두면 CNN에서 흔히 쓰는 전통적인 국소 풀링이 된다.  $s < z$ 이면 중첩 풀링이 된다. 우리는 네트워크 전체에서  $s=2, z=3$ 을 사용했다. 출력 크기가 같은 비중첩 방식  $s=2, z=2$ 와 비교했을 때 이 방식은 top-1과 top-5 오류율을 각각 0.4%와 0.3% 낮췄다. 학습 과정에서 중첩 풀링을 사용하는 모델은 과적합이 약간 더 어렵게 발생하는 경향도 관찰했다.

### 3.5 전체 아키텍처

이제 CNN의 전체 아키텍처를 설명한다. 그림 2와 같이 네트워크에는 가중치를 가진 레이어가 8개 있으며, 앞의 5개는 합성곱 레이어이고 나머지 3개는 완전 연결 레이어다. 마지막 완전 연결 레이어의 출력은 1,000개 클래스 라벨에 대한 분포를 만드는 1,000-way softmax에 입력된다. 네트워크는 다항 로지스틱 회귀 목적함수를 최대화하며, 이는 학습 예제별 예측 분포에서 정답 라벨의 로그 확률을 구해 그 평균을 최대화하는 것과 같다.

2, 4, 5번째 합성곱 레이어의 커널은 이전 레이어에서 같은 GPU에 있는 커널 맵에만 연결된다(그림 2 참조). 3번째 합성곱 레이어의 커널은 2번째 레이어의 모든 커널 맵에 연결된다. 완전 연결 레이어의 뉴런은 이전 레이어의 모든 뉴런에 연결된다. 응답 정규화 레이어는 첫 번째와 두 번째 합성곱 레이어 뒤에 위치한다. 3.4절에서 설명한 max-pooling 레이어는 두 응답 정규화 레이어 뒤와 다섯 번째 합성곱 레이어 뒤에 위치한다. 모든 합성곱 레이어와 완전 연결 레이어의 출력에는 ReLU 비선형성을 적용한다.

첫 번째 합성곱 레이어는  $224 \times 224 \times 3$  입력 이미지를  $11 \times 11 \times 3$  크기의 커널 96개와 stride 4픽셀로 필터링한다(이는 한 커널 맵에서 인접한 뉴런의 수용 영역 중심 사이의 거리다).

<sup>3</sup> 지면 제약으로 이 네트워크를 자세히 설명할 수는 없지만, 제공된 코드와 파라미터 파일에서 정확히 확인할 수 있다: <http://code.google.com/p/cuda-convnet/>.

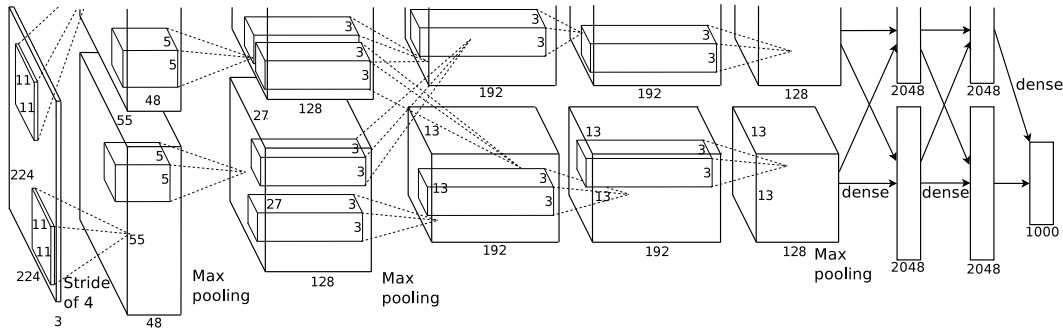


그림 2: 두 GPU 사이의 역할 분담을 명시적으로 보여주는 CNN 아키텍처. 한 GPU는 그림 위쪽의 레이어 부분을, 다른 GPU는 아래쪽 부분을 실행한다. GPU는 특정 레이어에서만 서로 통신한다. 네트워크 입력은 150,528차원이며, 이후 각 레이어의 뉴런 수는 253,440-186,624-64,896-64,896-43,264-4096-4096-1000이다.

두 번째 합성곱 레이어는 첫 번째 합성곱 레이어의 응답 정규화 및 풀링된 출력을 입력으로 받아  $5 \times 5 \times 48$  크기의 커널 256개로 필터링한다. 세 번째, 네 번째, 다섯 번째 합성곱 레이어 사이에는 중간 풀링이나 정규화 레이어가 없다. 세 번째 합성곱 레이어는 2번째 레이어의 정규화 및 풀링된 출력에 연결된  $3 \times 3 \times 256$  크기의 커널 384개를 갖는다. 네 번째 합성곱 레이어는  $3 \times 3 \times 192$  크기의 커널 384개를, 다섯 번째 합성곱 레이어는  $3 \times 3 \times 192$  크기의 커널 256개를 갖는다. 완전 연결 레이어는 각각 4096개의 뉴런을 갖는다.

## 4 과적합 감소

우리 신경망 아키텍처에는 6천만 개의 파라미터가 있다. ILSVRC의 1,000개 클래스는 각 학습 예제가 이미지에서 라벨로의 매핑에 약 10비트의 제약을 제공하게 하지만, 그 정도만으로는 이렇게 많은 파라미터를 심각한 과적합 없이 학습하기에 부족하다. 아래에서는 과적합을 줄이기 위해 사용한 두 가지 주요 방법을 설명한다.

### 4.1 데이터 증강

이미지 데이터에서 과적합을 줄이는 가장 쉽고 흔한 방법은 라벨을 보존하는 변환을 사용해 데이터셋을 인위적으로 늘리는 것이다(예: [25, 4, 5]). 우리는 서로 다른 두 가지 데이터 증강을 사용했다. 두 방법 모두 원본 이미지에서 변환된 이미지를 아주 적은 계산으로 생성할 수 있으므로, 변환 이미지를 디스크에 저장할 필요가 없다. 구현에서는 GPU가 이전 이미지 배치를 학습하는 동안 CPU의 Python 코드가 변환 이미지를 생성한다. 따라서 이러한 증강 방식은 실질적으로 추가 계산 비용이 거의 없다.

첫 번째 데이터 증강은 이미지의 평행 이동과 좌우 반전이다.  $256 \times 256$  이미지에서 무작위  $224 \times 224$  패치와 그 좌우 반전 이미지를 추출해 네트워크를 학습했다<sup>4</sup>. 이 방법은 학습 세트의 크기를 2048배 늘리지만, 생성된 학습 예제들은 당연히 서로 강하게 의존한다. 이 방법이 없으면 네트워크에 상당한 과적합이 발생해 훨씬 작은 네트워크를 사용해야 했을 것이다. 테스트 시에는 네트워크가  $224 \times 224$  패치 5개(네 모서리와 중앙)와 각각의 좌우 반전 이미지를 추출해 총 10개 패치에 대해 예측하고, softmax 레이어가 만든 10개의 예측을 평균한다.

두 번째 데이터 증강은 학습 이미지의 RGB 채널 강도를 변화시키는 것이다. 구체적으로 ImageNet 학습 세트 전체의 RGB 픽셀 값에 대해 PCA를 수행한다. 각 학습 이미지에는 구한 주성분에 해당 고유값과 평균 0, 표준편차 0.1의 가우시안 난수에 비례하는 크기를 곱해 더한다.

<sup>4</sup> 이 때문에 그림 2의 입력 이미지는  $224 \times 224 \times 3$  차원이다.

따라서 각 RGB 이미지 픽셀  $I_{xy} = [I_R(x,y), I_G(x,y), I_B(x,y)]^T$ 에 아래의 값을 더한다.

$$[p_1, p_2, p_3][\alpha_1 \lambda_1, \alpha_2 \lambda_2, \alpha_3 \lambda_3]^T$$

여기서  $p_i$ 와  $\lambda_i$ 는 각각 RGB 픽셀 값의  $3 \times 3$  공분산 행렬에서  $i$ 번째 고유벡터와 고유값이며,  $\alpha_i$ 는 앞서 말한 난수다. 특정 학습 이미지의 모든 픽셀에는 각  $\alpha_i$ 를 한 번만 뽑아 동일하게 사용하고, 그 이미지가 다시 학습에 사용될 때 새로 뽑는다. 이 방법은 자연 이미지의 중요한 특성, 즉 조명의 강도와 색이 변해도 객체의 정체성은 변하지 않는다는 성질을 근사적으로 반영한다. 이 방식은 top-1 오류율을 1% 이상 낮춘다.

#### 4.2 드롭아웃

여러 서로 다른 모델의 예측을 결합하는 것은 테스트 오류를 줄이는 매우 성공적인 방법이지만 [1, 3], 이미 학습에 며칠이 걸리는 큰 신경망에는 비용이 너무 크다. 그러나 학습 비용을 약 두 배만 늘리면서 모델 결합과 비슷한 효과를 얻는 매우 효율적인 방법이 있다. 최근 제안된 “dropout” [10]은 각 은닉 뉴런의 출력을 확률 0.5로 0으로 만드는 방식이다. 이렇게 “드롭된” 뉴런은 순전파에 기여하지 않고 역전파에도 참여하지 않는다. 따라서 입력이 들어올 때마다 신경망은 서로 다른 아키텍처를 샘플링하지만, 이 모든 아키텍처는 가중치를 공유한다. 이 기법은 뉴런 사이의 복잡한 공동 적응(co-adaptation)을 줄인다. 한 뉴런이 특정 다른 뉴런의 존재에 의존할 수 없기 때문에, 다양한 무작위 뉴런 부분집합과 함께 사용할 수 있는 더 강건한 특징을 학습하도록 강제된다. 테스트 시에는 모든 뉴런을 사용하되 출력을 0.5배 하는데, 이는 지수적으로 많은 dropout 네트워크가 만드는 예측 분포의 기하평균을 취하는 것을 합리적으로 근사한다.

그림 2의 처음 두 완전 연결 레이어에 dropout을 사용했다. dropout이 없으면 네트워크는 상당한 과적합을 보인다. dropout을 사용하면 수렴에 필요한 반복 횟수는 대략 두 배가 된다.

#### 5 학습 세부사항

우리는 배치 크기 128, 모멘텀 0.9, weight decay 0.0005를 사용한 확률적 경사하강법(SGD)으로 모델을 학습했다. 이 작은 weight decay가 모델 학습에 중요하다는 것을 확인했다. 즉, 여기서 weight decay는 단순한 정규화만이 아니라 학습 오류 자체를 낮춘다. 가중치  $w$ 의 업데이트 규칙은 다음과 같다.

$$v_{i+1} := 0.9 \cdot v_i - 0.0005 \cdot \epsilon \cdot w_i - \epsilon \cdot \left\langle \frac{\partial L}{\partial w} \Big|_{w_i} \right\rangle_{D_i}$$

$$w_{i+1} := w_i + v_{i+1}$$

여기서  $i$ 는 반복 인덱스,  $v$ 는 모멘텀 변수,  $\epsilon$ 은 학습률이다. 식의 미분 항은  $i$ 번째 배치  $D_i$ 에서  $w=w_i$ 일 때 목적함수를  $w$ 로 미분한 값의 평균이다.

각 레이어의 가중치는 평균 0, 표준편차 0.01인 가우시안 분포에서 초기화했다. 두 번째, 네 번째, 다섯 번째 합성곱 레이어와 완전 연결 은닉 레이어의 뉴런 bias는 상수 1로 초기화했다. 이 초기화는 ReLU에 양의 입력을 제공해 학습 초기 단계를 빠르게 한다. 나머지 레이어의 뉴런 bias는 상수 0으로 초기화했다.

모든 레이어에 같은 학습률을 사용했고 학습 중 수동으로 조정했다. 현재 학습률에서 검증 오류율이 더 이상 개선되지 않으면 학습률을 10으로 나누는 휴리스틱을 사용했다. 초기 학습률은 0.01이었다.

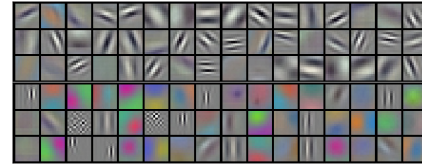


그림 3:  $224 \times 224 \times 3$  입력 이미지에 대해 첫 번째 합성곱 레이어가 학습한  $11 \times 11 \times 3$  크기의 합성곱 커널 96개. 위쪽 48개 커널은 GPU 1에서, 아래쪽 48개 커널은 GPU 2에서 학습되었다. 자세한 내용은 6.1절을 참조하라.

학습 종료 전까지 학습률을 세 번 낮췄다. 120만 장의 학습 세트를 약 90회 순회했으며, NVIDIA GTX 580 3GB GPU 두 장에서 5~6일이 걸렸다.

## 6 결과

ILSVRC-2010 결과는 표 1에 요약되어 있다. 우리 네트워크의 테스트 세트 top-1과 top-5 오류율은 각각 37.5%와 17.0%(주 5)였다. ILSVRC-2010 대회 당시 최고 성능은 서로 다른 특징으로 학습한 6개의 sparse-coding 모델의 예측을 평균한 방법 [2]으로 47.1%와 28.2%였고, 이후 발표된 최고 결과는 두 종류의 조밀 샘플 특징에서 계산한 Fisher Vector(FV)를 사용해 학습한 두 분류기의 예측을 평균한 방법 [24]으로 45.7%와 25.7%였다.

우리 모델을 ILSVRC-2012 대회에도 출전시켰으며 결과는 표 2에 제시한다. ILSVRC-2012의 테스트 세트 라벨은 공개되지 않았기 때문에 시도한 모든 모델에 대해 테스트 오류율을 보고할 수는 없다. 이 문단의 나머지에서는 검증 오류율과 테스트 오류율을 같은 의미로 사용한다. 경험상 두 값의 차이가 0.1%를 넘지 않았기 때문이다(표 2 참조). 이 논문에서 설명한 단일 CNN은 top-5 오류율 18.2%를 달성한다.

| Model                    | Top-1        | Top-5        |
|--------------------------|--------------|--------------|
| <i>Sparse coding [2]</i> | 47.1%        | 28.2%        |
| <i>SIFT + FVs [24]</i>   | 45.7%        | 25.7%        |
| CNN                      | <b>37.5%</b> | <b>17.0%</b> |

표 1: ILSVRC-2010 테스트 세트 결과 비교. 이탤릭체는 다른 연구의 최고 결과를 뜻한다.

비슷한 CNN 5개의 예측을 평균하면 오류율은 16.4%가 된다. ImageNet Fall 2011 전체(1,500만 이미지, 22,000 범주)를 분류하도록 마지막 풀링 레이어 위에 여섯 번째 합성곱 레이어를 추가한 CNN 하나를 학습한 뒤 ILSVRC-2012에 “fine-tuning”하면 오류율은 16.6%가 된다. Fall 2011 전체 데이터로 사전 학습한 CNN 2개의 예측과 앞서 말한 CNN 5개의 예측을 평균하면 15.3%를 달성한다. 대회 2위는 여러 종류의 조밀 샘플 특징에서 계산한 FV를 사용한 여러 분류기의 예측을 평균하는 방법 [7]으로 26.2%의 오류율을 기록했다.

마지막으로 10,184개 범주와 890만 장의 이미지를 포함하는 ImageNet Fall 2009 버전에 대한 오류율도 보고한다. 이 데이터셋에서는 문헌의 관례를 따라 이미지 절반을 학습에, 절반을 테스트에 사용한다. 확립된 테스트 세트가 없으므로 우리의 분할은 이전 연구자들이 사용한 분할과 다를 수밖에 없지만, 결과에는 큰 영향을 주지 않는다. 이 데이터셋의 top-1과 top-5 오류율은 각각 67.4%와

| Model                 | Top-1 (val) | Top-5 (val) | Top-5 (test) |
|-----------------------|-------------|-------------|--------------|
| <i>SIFT + FVs [7]</i> | —           | —           | 26.2%        |
| 1 CNN                 | 40.7%       | 18.2%       | —            |
| 5 CNNs                | 38.1%       | 16.4%       | <b>16.4%</b> |
| 1 CNN*                | 39.0%       | 16.6%       | —            |
| 7 CNNs*               | 36.7%       | 15.4%       | <b>15.3%</b> |

표 2: ILSVRC-2012 검증 및 테스트 세트 오류율 비교. 이탤릭체는 다른 연구의 최고 결과를 뜻한다. 별표(\*)가 있는 모델은 ImageNet 2011 Fall 전체 데이터를 분류하도록 “사전 학습”되었다. 자세한 내용은 6절을 참조하라.

40.9%였다. 이는 마지막 풀링 레이어 위에 여섯 번째 합성곱 레이어를 추가한 앞의 네트워크로 얻은 결과다. 이 데이터셋에 대해 이전에 발표된 최고 결과는 78.1%와 60.9%였다 [19].

### 6.1 정성적 평가

그림 3은 네트워크의 데이터에 직접 연결된 두 레이어가 학습한 합성곱 커널을 보여준다. 네트워크는 다양한 주파수 및 방향 선택적 커널과 여러 색상 블롭(blob)을 학습했다. 3.5절에서 설명한 제한된 연결성의 결과로 두 GPU가 서로 전문화되는 현상에 주목하라. GPU 1의 커널은 대체로 색상에 무관한 반면, GPU 2의 커널은 대체로 특정 색상에 민감하다. 이러한 전문화는 모든 학습 실행에서 나타나며, GPU 번호가 바뀌는 경우를 제외하면 특정 무작위 가중치 초기화와 무관하다.

주 5: 4.1절에서 설명한 10개 패치의 예측 평균을 사용하지 않으면 오류율은 각각 39.0%와 18.3%다.

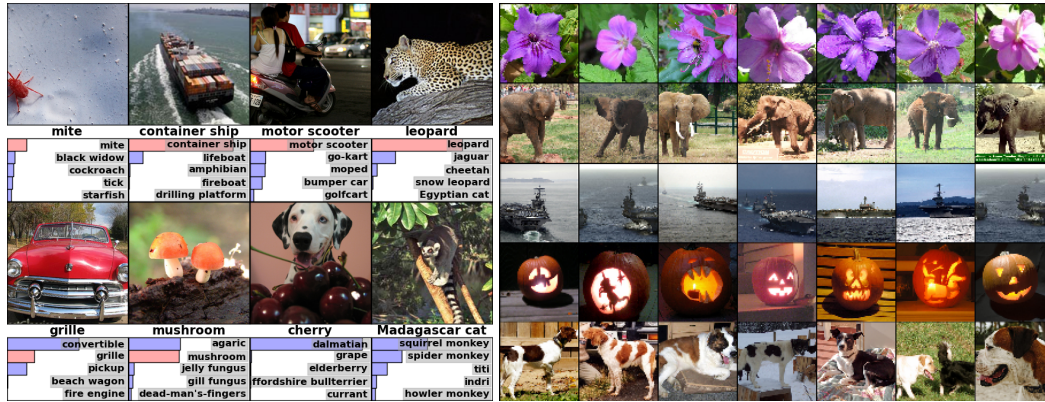


그림 4: (왼쪽) ILSVRC-2010 테스트 이미지 8장과 모델이 가장 가능성이 높다고 판단한 5개 라벨. 각 이미지 아래에는 정답 라벨이 표시되며, 정답이 top-5 안에 있으면 그 라벨에 할당된 확률도 빨간 막대로 표시된다. (오른쪽) 첫 번째 열에는 ILSVRC-2010 테스트 이미지 5장이 있으며, 나머지 열에는 마지막 은닉 레이어의 특징 벡터가 해당 테스트 이미지의 특징 벡터와 유클리드 거리가 가장 가까운 학습 이미지 6장이 표시된다.

그림 4의 왼쪽 패널에서는 테스트 이미지 8장에 대해 top-5 예측을 계산해 네트워크가 무엇을 학습했는지 정성적으로 평가한다. 왼쪽 위의 진드기처럼 중심에서 벗어난 객체도 네트워크가 인식할 수 있다는 점에 주목하라. 대부분의 top-5 라벨은 합리적으로 보인다. 예를 들어 표범에 대해서는 다른 종류의 고양이들만 가능한 라벨로 판단한다. 어떤 경우(grille, cherry)에는 사진의 초점이 무엇인지 자체가 실제로 모호하다.

네트워크의 시각적 지식을 살펴보는 또 다른 방법은 이미지가 마지막 4096차원 은닉 레이어에서 만들어내는 특징 활성값을 보는 것이다. 두 이미지의 특징 활성 벡터 사이 유클리드 거리가 작다면, 신경망의 상위 레벨은 두 이미지를 비슷하다고 판단한다고 볼 수 있다. 그림 4는 테스트 세트의 이미지 5장과, 이 기준으로 각각 가장 유사한 학습 이미지 6장을 보여준다. 픽셀 수준에서는 검색된 학습 이미지가 첫 번째 열의 질의 이미지와 일반적으로 L2 거리가 가깝지 않다는 점에 주목하라. 예를 들어 검색된 개와 코끼리는 다양한 자세로 나타난다. 더 많은 테스트 이미지에 대한 결과는 보충 자료에 제시한다.

4096차원 실수 벡터 두 개 사이의 유클리드 거리로 유사도를 계산하는 것은 비효율적이지만, auto-encoder를 학습해 이 벡터를 짧은 이진 코드로 압축하면 효율적으로 만들 수 있다. 이는 원시 픽셀에 auto-encoder를 직접 적용하는 방법 [14]보다 훨씬 좋은 이미지 검색 방법이 될 것이다. 원시 픽셀 기반 방법은 이미지 라벨을 사용하지 않으므로 의미가 비슷한지와 관계없이 비슷한 에지 패턴을 가진 이미지를 검색하는 경향이 있기 때문이다.

## 7 논의

우리의 결과는 크고 깊은 합성곱 신경망이 순수한 지도학습만으로도 매우 어려운 데이터셋에서 기록적인 성능을 달성할 수 있음을 보여준다. 특히 합성곱 레이어 하나만 제거해도 네트워크 성능이 저하된다는 점이 중요하다. 예를 들어 중간 레이어 중 하나를 제거하면 네트워크의 top-1 성능이 약 2% 나빠진다. 따라서 이 결과를 얻는 데 깊이는 실제로 중요하다.

실험을 단순하게 하기 위해 비지도 사전 학습은 사용하지 않았지만, 특히 라벨 데이터의 양은 그대로인 상태에서 충분한 계산 능력으로 네트워크 크기를 크게 늘릴 수 있다면 도움이 될 것으로 예상된다. 지금까지는 네트워크를 더 크게 만들고 더 오래 학습할수록 결과가 좋아졌지만, 인간 시각 시스템의 infero-temporal pathway에 맞추기까지는 여전히 엄청난 규모의 차이가 남아 있다. 궁극적으로는 정적 이미지에서는 없거나 훨씬 덜 분명한 유용한 정보를 시간 구조가 제공하는 비디오 시퀀스에 매우 크고 깊은 합성곱 신경망을 적용하고자 한다.



## 참고문헌

- [1] R.M. Bell and Y. Koren. Lessons from the netflix prize challenge. *ACM SIGKDD Explorations Newsletter*, 9(2):75–79, 2007.
- [2] A. Berg, J. Deng, and L. Fei-Fei. Large scale visual recognition challenge 2010. [www.image-net.org/challenges](http://www.image-net.org/challenges). 2010.
- [3] L. Breiman. Random forests. *Machine learning*, 45(1):5–32, 2001.
- [4] D. Cireřan, U. Meier, and J. Schmidhuber. Multi-column deep neural networks for image classification. *Arxiv preprint arXiv:1202.2745*, 2012.
- [5] D.C. Cireřan, U. Meier, J. Masci, L.M. Gambardella, and J. Schmidhuber. High-performance neural networks for visual object classification. *Arxiv preprint arXiv:1102.0183*, 2011.
- [6] J. Deng, W. Dong, R. Socher, L.-J. Li, K. Li, and L. Fei-Fei. ImageNet: A Large-Scale Hierarchical Image Database. In *CVPR09*, 2009.
- [7] J. Deng, A. Berg, S. Satheesh, H. Su, A. Khosla, and L. Fei-Fei. *ILSVRC-2012*, 2012. URL <http://www.image-net.org/challenges/LSVRC/2012/>.
- [8] L. Fei-Fei, R. Fergus, and P. Perona. Learning generative visual models from few training examples: An incremental bayesian approach tested on 101 object categories. *Computer Vision and Image Understanding*, 106(1):59–70, 2007.
- [9] G. Griffin, A. Holub, and P. Perona. Caltech-256 object category dataset. Technical Report 7694, California Institute of Technology, 2007. URL <http://authors.library.caltech.edu/7694>.
- [10] G.E. Hinton, N. Srivastava, A. Krizhevsky, I. Sutskever, and R.R. Salakhutdinov. Improving neural networks by preventing co-adaptation of feature detectors. *arXiv preprint arXiv:1207.0580*, 2012.
- [11] K. Jarrett, K. Kavukcuoglu, M. A. Ranzato, and Y. LeCun. What is the best multi-stage architecture for object recognition? In *International Conference on Computer Vision*, pages 2146–2153. IEEE, 2009.
- [12] A. Krizhevsky. Learning multiple layers of features from tiny images. Master’s thesis, Department of Computer Science, University of Toronto, 2009.
- [13] A. Krizhevsky. Convolutional deep belief networks on cifar-10. *Unpublished manuscript*, 2010.
- [14] A. Krizhevsky and G.E. Hinton. Using very deep autoencoders for content-based image retrieval. In *ESANN*, 2011.
- [15] Y. Le Cun, B. Boser, J.S. Denker, D. Henderson, R.E. Howard, W. Hubbard, L.D. Jackel, et al. Hand-written digit recognition with a back-propagation network. In *Advances in neural information processing systems*, 1990.
- [16] Y. LeCun, F.J. Huang, and L. Bottou. Learning methods for generic object recognition with invariance to pose and lighting. In *Computer Vision and Pattern Recognition, 2004. CVPR 2004. Proceedings of the 2004 IEEE Computer Society Conference on*, volume 2, pages II–97. IEEE, 2004.
- [17] Y. LeCun, K. Kavukcuoglu, and C. Farabet. Convolutional networks and applications in vision. In *Circuits and Systems (ISCAS), Proceedings of 2010 IEEE International Symposium on*, pages 253–256. IEEE, 2010.
- [18] H. Lee, R. Grosse, R. Ranganath, and A.Y. Ng. Convolutional deep belief networks for scalable unsupervised learning of hierarchical representations. In *Proceedings of the 26th Annual International Conference on Machine Learning*, pages 609–616. ACM, 2009.
- [19] T. Mensink, J. Verbeek, F. Perronnin, and G. Csorba. Metric Learning for Large Scale Image Classification: Generalizing to New Classes at Near-Zero Cost. In *ECCV - European Conference on Computer Vision*, Florence, Italy, October 2012.
- [20] V. Nair and G. E. Hinton. Rectified linear units improve restricted boltzmann machines. In *Proc. 27th International Conference on Machine Learning*, 2010.
- [21] N. Pinto, D.D. Cox, and J.J. DiCarlo. Why is real-world visual object recognition hard? *PLoS computational biology*, 4(1):e27, 2008.
- [22] N. Pinto, D. Doukhan, J.J. DiCarlo, and D.D. Cox. A high-throughput screening approach to discovering good forms of biologically inspired visual representation. *PLoS computational biology*, 5(11):e1000579, 2009.
- [23] B.C. Russell, A. Torralba, K.P. Murphy, and W.T. Freeman. Labelme: a database and web-based tool for image annotation. *International journal of computer vision*, 77(1):157–173, 2008.
- [24] J. Sánchez and F. Perronnin. High-dimensional signature compression for large-scale image classification. In *Computer Vision and Pattern Recognition (CVPR), 2011 IEEE Conference on*, pages 1665–1672. IEEE, 2011.
- [25] P.Y. Simard, D. Steinkraus, and J.C. Platt. Best practices for convolutional neural networks applied to visual document analysis. In *Proceedings of the Seventh International Conference on Document Analysis and Recognition*, volume 2, pages 958–962, 2003.
- [26] S.C. Turaga, J.F. Murray, V. Jain, F. Roth, M. Helmstaedter, K. Briggman, W. Denk, and H.S. Seung. Convolutional networks can learn to generate affinity graphs for image segmentation. *Neural Computation*, 22(2):511–538, 2010.